

AgroAid - Codigo de features para copiar

Generado el 2026-09-27

Feature	Crear	Modificar
Mapa	routers/mapa.py, frontend/mapa.html	core/database.py, app.py
Pre-vuelo	routers/prevuelo.py	core/database.py, app.py
Widget	frontend/widget.js, frontend/widget-demo.html	app.py
Recetas	routers/recetas.py, core/pdf_receta.py	core/database.py, app.py

Como pegar estos cambios

Este PDF contiene codigo listo para transcribir manualmente.

- Los archivos nuevos van completos.
- app.py y core/database.py van como snippets de modificacion, porque app.py ya es grande.
- El codigo esta adaptado al proyecto actual: usa `require_any/require_admin`, los helpers de `core.database`, `dispatch_webhook` y `core.clima.get_clima` tal como existen hoy.
- Despues de copiar, reinicia la API para que `ensure_schema` cree las tablas nuevas.

core/database.py - agregar schema de features

```
# 1) Agregar esta constante antes de ensure_schema()

FEATURE_SCHEMA_SQL = """
CREATE TABLE IF NOT EXISTS eventos_fitosanitarios (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER REFERENCES tenants(id),
    user_id INTEGER REFERENCES users(id),
    lat DOUBLE PRECISION NOT NULL,
    lon DOUBLE PRECISION NOT NULL,
    tipo TEXT NOT NULL,
    descripcion TEXT,
    nivel_riesgo TEXT,
    radio_km DOUBLE PRECISION DEFAULT 5,
    verificado BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_eventos_fitosanitarios_tenant
ON eventos_fitosanitarios(tenant_id);

CREATE INDEX IF NOT EXISTS idx_eventos_fitosanitarios_geo
ON eventos_fitosanitarios(lat, lon);

CREATE TABLE IF NOT EXISTS pre_aplicaciones (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER REFERENCES tenants(id),
    user_id INTEGER REFERENCES users(id),
    lat DOUBLE PRECISION NOT NULL,
    lon DOUBLE PRECISION NOT NULL,
    producto TEXT NOT NULL,
    dosis_l_ha DOUBLE PRECISION,
    cultivo TEXT,
    distancia_agua_m DOUBLE PRECISION,
    hora_aplicacion TIMESTAMPTZ,
    tipo_equipo TEXT,
    resultado_global TEXT NOT NULL,
    detalle JSONB NOT NULL,
    pdf_path TEXT,
    entry_hash TEXT,
    prev_hash TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_pre_aplicaciones_tenant
ON pre_aplicaciones(tenant_id);

CREATE TABLE IF NOT EXISTS recetas_agronomicas (
    id SERIAL PRIMARY KEY,
    tenant_id INTEGER REFERENCES tenants(id),
    user_id INTEGER REFERENCES users(id),
    consulta_id INTEGER REFERENCES consultas(id),
    numero_receta TEXT NOT NULL UNIQUE,
    producto TEXT NOT NULL,
    principio_activo TEXT,
    cultivo TEXT NOT NULL,
    lote TEXT,
    superficie_ha DOUBLE PRECISION,
    dosis TEXT NOT NULL,
    volumen_agua TEXT,
    fecha_aplicacion DATE,
    observaciones TEXT,
    nivel_riesgo TEXT,
    estado TEXT NOT NULL DEFAULT 'borrador',
    pdf_path TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```

);

CREATE INDEX IF NOT EXISTS idx_recetas_agronomicas_tenant
    ON recetas_agronomicas(tenant_id);

CREATE INDEX IF NOT EXISTS idx_recetas_agronomicas_consulta
    ON recetas_agronomicas(consulta_id);
"""

# 2) Dentro de ensure_schema(), justo despues del loop que ejecuta schema.sql:
#
#     for stmt in stmts:
#         cur.execute(stmt)
#
# agregar este segundo loop:

feature_stmts = [
    s.strip()
    for s in FEATURE_SCHEMA_SQL.split(";")
    if s.strip()
]

for stmt in feature_stmts:
    cur.execute(stmt)

```

routers/mapa.py

```
from __future__ import annotations

from typing import Literal

from fastapi import APIRouter, Depends, HTTPException, Query
from pydantic import BaseModel, Field

from core.database import db_fetch_all, db_fetch_one
from core.deps import require_admin, require_any
from core.webhook import dispatch_webhook

router = APIRouter(prefix="/api/mapa", tags=["Mapa fitosanitario"])

TipoEvento = Literal["plaga", "enfermedad", "contaminacion", "incendio"]

RIESGO_POR_TIPO = {
    "plaga": "ALTO",
    "enfermedad": "ALTO",
    "contaminacion": "CRITICO",
    "incendio": "CRITICO",
}

class EventoCreate(BaseModel):
    lat: float = Field(..., ge=-90, le=90)
    lon: float = Field(..., ge=-180, le=180)
    tipo: TipoEvento
    descripcion: str | None = Field(default=None, max_length=2000)
    nivel_riesgo: str | None = None
    radio_km: float = Field(default=5, gt=0, le=200)

def _tenant_id(current_user: dict) -> int:
    return int(current_user.get("tenant_id") or 1)

def _user_id(current_user: dict) -> int | None:
    raw = current_user.get("id")
    return int(raw) if raw is not None else None

@router.post("/eventos")
async def crear_evento(
    req: EventoCreate,
    current_user: dict = Depends(require_any),
):
    tenant_id = _tenant_id(current_user)
    user_id = _user_id(current_user)
    nivel = (req.nivel_riesgo or RIESGO_POR_TIPO[req.tipo]).upper()

    evento = db_fetch_one(
        """
        INSERT INTO eventos_fitosanitarios
            (tenant_id, user_id, lat, lon, tipo, descripcion,
             nivel_riesgo, radio_km)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
        RETURNING id, tenant_id, user_id, lat, lon, tipo, descripcion,
                  nivel_riesgo, radio_km, verificado, created_at
        """,
        (
            tenant_id,
            user_id,
            req.lat,
            req.lon,
        )
    )
```

```

        req.tipo,
        req.descripcion,
        nivel,
        req.radio_km,
    ),
)

if nivel in {"ALTO", "CRITICO"}:
    await dispatch_webhook(
        tenant_id,
        "evento_fitosanitario",
        {
            "evento_id": evento["id"],
            "tipo": evento["tipo"],
            "nivel_riesgo": evento["nivel_riesgo"],
            "lat": evento["lat"],
            "lon": evento["lon"],
            "descripcion": evento["descripcion"],
        },
    )

return evento

@router.get("/eventos")
async def listar_eventos(
    lat: float = Query(..., ge=-90, le=90),
    lon: float = Query(..., ge=-180, le=180),
    radio_km: float = Query(default=50, gt=0, le=500),
    current_user: dict = Depends(require_any),
):
    tenant_id = _tenant_id(current_user)

    return db_fetch_all(
        """
        SELECT *
        FROM (
            SELECT
                e.id, e.tenant_id, e.user_id, e.lat, e.lon, e.tipo,
                e.descripcion, e.nivel_riesgo, e.radio_km, e.verificado,
                e.created_at,
                6371 * acos(
                    LEAST(
                        1,
                        GREATEST(
                            -1,
                            cos(radians(%s)) * cos(radians(e.lat)) *
                            cos(radians(e.lon) - radians(%s)) +
                            sin(radians(%s)) * sin(radians(e.lat))
                        )
                    ) AS distancia_km
                FROM eventos_fitosanitarios e
                WHERE e.tenant_id = %s
            ) q
            WHERE q.distancia_km <= %s
            ORDER BY q.distancia_km ASC
            LIMIT 200
        """
        (lat, lon, lat, tenant_id, radio_km),
    )

@router.put("/eventos/{evento_id}/verificar")
async def verificar_evento(
    evento_id: int,
    current_user: dict = Depends(require_admin),
):

```

```

tenant_id = _tenant_id(current_user)
evento = db_fetch_one(
    """
    UPDATE eventos_fitosanitarios
    SET verificado = TRUE
    WHERE id = %s
    AND tenant_id = %s
    RETURNING id, tenant_id, user_id, lat, lon, tipo, descripcion,
              nivel_riesgo, radio_km, verificado, created_at
    """
    (evento_id, tenant_id),
)

if not evento:
    raise HTTPException(status_code=404, detail="Evento no encontrado")

return evento

```

frontend/mapa.html

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>AgroAid - Mapa fitosanitario</title>
  <link
    rel="stylesheet"
    href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"
  />
  <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<style>
  :root {
    color-scheme: light;
    font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  }
  body {
    margin: 0;
    background: #f8fafc;
    color: #0f172a;
  }
  header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 12px;
    padding: 12px 16px;
    background: #14532d;
    color: white;
  }
  header h1 {
    margin: 0;
    font-size: 18px;
  }
  .token-box {
    display: flex;
    gap: 8px;
    align-items: center;
  }
  .token-box input {
    width: min(46vw, 360px);
    padding: 8px;
    border: 1px solid #86efac;
    border-radius: 6px;
  }
  button {
    border: 0;
    border-radius: 6px;
    padding: 9px 12px;
    background: #16a34a;
    color: white;
    font-weight: 700;
    cursor: pointer;
  }
  #map {
    height: calc(100vh - 58px);
    width: 100%;
  }
  .panel {
    position: fixed;
    right: 14px;
    top: 74px;
    z-index: 1000;
    width: min(360px, calc(100vw - 28px));
    background: white;
```

```

border: 1px solid #cbd5e1;
border-radius: 8px;
box-shadow: 0 18px 50px rgba(15, 23, 42, 0.2);
padding: 14px;
display: none;
}
.panel.open {
display: block;
}
.panel h2 {
margin: 0 0 10px;
font-size: 16px;
}
label {
display: grid;
gap: 5px;
margin: 8px 0;
font-size: 13px;
font-weight: 700;
}
select,
textarea,
input[type="number"] {
width: 100%;
box-sizing: border-box;
border: 1px solid #cbd5e1;
border-radius: 6px;
padding: 8px;
font: inherit;
}
textarea {
min-height: 86px;
resize: vertical;
}
.actions {
display: flex;
justify-content: flex-end;
gap: 8px;
margin-top: 10px;
}
.secondary {
background: #e2e8f0;
color: #0f172a;
}
.marker-dot {
width: 16px;
height: 16px;
border-radius: 50%;
border: 2px solid white;
box-shadow: 0 2px 8px rgba(15, 23, 42, 0.45);
}
@media (max-width: 720px) {
header {
align-items: stretch;
flex-direction: column;
}
.token-box input {
width: 100%;
}
}
</style>
</head>
<body>
<header>
<h1>Mapa colaborativo fitosanitario</h1>
<div class="token-box">
<input id="token" placeholder="JWT o API Key" />
<button id="guardarToken">Guardar</button>

```

```

</div>
</header>

<div id="map"></div>

<form id="panel" class="panel">
  <h2>Reportar evento</h2>
  <input id="lat" type="hidden" />
  <input id="lon" type="hidden" />
  <label>
    Tipo
    <select id="tipo" required>
      <option value="plaga">Plaga</option>
      <option value="enfermedad">Enfermedad</option>
      <option value="contaminacion">Contaminacion</option>
      <option value="incendio">Incendio</option>
    </select>
  </label>
  <label>
    Radio estimado (km)
    <input id="radio" type="number" min="1" max="200" step="1" value="5" />
  </label>
  <label>
    Descripcion
    <textarea id="descripcion" required></textarea>
  </label>
  <div class="actions">
    <button type="button" class="secondary" id="cancelar">Cancelar</button>
    <button type="submit">Enviar</button>
  </div>
</form>

<script>
const tokenInput = document.getElementById("token");
const savedToken = localStorage.getItem("agroaid_token") || "";
tokenInput.value = savedToken;

document.getElementById("guardarToken").addEventListener("click", () => {
  localStorage.setItem("agroaid_token", tokenInput.value.trim());
  cargarEventos();
});

const map = L.map("map").setView([-34.6, -58.4], 6);
const markers = L.layerGroup().addTo(map);
let currentCenter = { lat: -34.6, lon: -58.4 };

L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
  maxZoom: 19,
  attribution: "&copy; OpenStreetMap",
}).addTo(map);

const colors = {
  plaga: "#dc2626",
  enfermedad: "#f97316",
  contaminacion: "#ca8a04",
  incendio: "#1e3a8a",
};

function authHeaders() {
  const value = tokenInput.value.trim();
  if (!value) return {};
  if (value.startsWith("ags_")) return { "X-API-Key": value };
  return { Authorization: `Bearer ${value}` };
}

function iconFor(tipo) {
  const color = colors[tipo] || "#334155";
  return L.divIcon({

```

```

        className: "",
        html: `

${ev.tipo}</strong><br>` +
                `Riesgo: ${ev.nivel_riesgo || "-"}<br>` +
                `Distancia: ${Number(ev.distancia_km || 0).toFixed(1)} km<br>` +
                `${ev.descripcion || ""}`
            )
            .addTo(markers);
    });
}

function usarUbicacion(pos) {
    currentCenter = {
        lat: pos.coords.latitude,
        lon: pos.coords.longitude,
    };
    map.setView([currentCenter.lat, currentCenter.lon], 11);
    cargarEventos();
}

if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(usarUbicacion, cargarEventos);
} else {
    cargarEventos();
}

map.on("click", (event) => {
    document.getElementById("lat").value = event.latlng.lat;
    document.getElementById("lon").value = event.latlng.lng;
    document.getElementById("panel").classList.add("open");
});

document.getElementById("cancelar").addEventListener("click", () => {
    document.getElementById("panel").classList.remove("open");
});

document.getElementById("panel").addEventListener("submit", async (event) => {
    event.preventDefault();
    const body = {
        lat: Number(document.getElementById("lat").value),
        lon: Number(document.getElementById("lon").value),
        tipo: document.getElementById("tipo").value,
        radio_km: Number(document.getElementById("radio").value || 5),
        descripcion: document.getElementById("descripcion").value.trim(),
    };

    const res = await fetch("/api/mapa/eventos", {
        method: "POST",
        headers: {


```

```
        "Content-Type": "application/json",
        ...authHeaders(),
    },
    body: JSON.stringify(body),
});

if (!res.ok) {
    alert("No se pudo crear el evento");
    return;
}

event.target.reset();
document.getElementById("radio").value = 5;
document.getElementById("panel").classList.remove("open");
cargarEventos();
});
</script>
</body>
</html>
```

routers/prevuelo.py

```
from __future__ import annotations

import hashlib
import json
import pathlib
from datetime import datetime, timezone

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import FileResponse
from psycpg2.extras import RealDictCursor
from pydantic import BaseModel, Field
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle

from core.clima import get_clima
from core.database import db_fetch_one, get_conn
from core.deps import require_any

router = APIRouter(prefix="/api", tags=["Pre-aplicacion"])

class PreAplicacionRequest(BaseModel):
    lat: float = Field(..., ge=-90, le=90)
    lon: float = Field(..., ge=-180, le=180)
    producto: str
    dosis_l_ha: float | None = Field(default=None, ge=0)
    cultivo: str | None = None
    distancia_agua_m: float = Field(..., ge=0)
    hora_aplicacion: datetime | None = None
    tipo_equipo: str = "terrestre"
    epp_disponible: list[str] = []

def _tenant_id(current_user: dict) -> int:
    return int(current_user.get("tenant_id") or 1)

def _user_id(current_user: dict) -> int | None:
    raw = current_user.get("id")
    return int(raw) if raw is not None else None

def _tenant_slug(current_user: dict) -> str:
    return (current_user.get("tenant_slug") or "default").strip() or "default"

def _normalizar_epp(items: list[str]) -> set[str]:
    normalizados = set()
    for item in items or []:
        value = item.strip().lower()
        normalizados.add(value)
        if value == "mascara":
            normalizados.add("máscara")
        if value == "máscara":
            normalizados.add("mascara")
    return normalizados

async def evaluar_pre_aplicacion(req: PreAplicacionRequest) -> dict:
    clima = await get_clima(req.lat, req.lon)
    viento = float(clima.get("wind") or 0)
    lluvia = float(clima.get("rain") or 0)
```

```

checks = []
go = True

if viento > 15:
    checks.append({
        "check": "viento",
        "resultado": "NO-GO",
        "detalle": f"Viento {viento:.1f} km/h - maximo permitido 15 km/h",
    })
    go = False
else:
    checks.append({
        "check": "viento",
        "resultado": "GO",
        "detalle": f"Viento {viento:.1f} km/h - dentro del limite",
    })

if lluvia > 0:
    checks.append({
        "check": "lluvia",
        "resultado": "NO-GO",
        "detalle": f"Precipitacion actual {lluvia:.1f} mm - no aplicar",
    })
    go = False
else:
    checks.append({
        "check": "lluvia",
        "resultado": "GO",
        "detalle": "Sin precipitacion actual registrada",
    })

if req.distancia_agua_m < 50:
    checks.append({
        "check": "distancia_agua",
        "resultado": "NO-GO",
        "detalle": f"Solo {req.distancia_agua_m:.0f} m del agua - minimo 50 m",
    })
    go = False
else:
    checks.append({
        "check": "distancia_agua",
        "resultado": "GO",
        "detalle": f"{req.distancia_agua_m:.0f} m a agua - cumple minimo",
    })

epp = _normalizar_epp(req.epp_disponible)
epp_minimo = ["guantes", "mascara"]
faltante = [item for item in epp_minimo if item not in epp]
if faltante:
    checks.append({
        "check": "epp",
        "resultado": "NO-GO",
        "detalle": f"Falta EPP: {'', '.join(faltante)}",
    })
    go = False
else:
    checks.append({
        "check": "epp",
        "resultado": "GO",
        "detalle": "EPP minimo disponible",
    })

equipo = (req.tipo_equipo or "terrestre").lower()
factor_equipo = 1.5 if equipo == "drone" else 1.0
deriva_m = viento * 0.5 * factor_equipo
if deriva_m > req.distancia_agua_m * 0.3:
    checks.append({
        "check": "deriva",
    })

```

```

        "resultado": "PRECAUCION",
        "detalle": (
            f"Deriva estimada {deriva_m:.0f} m - revisar direccion de viento"
        ),
    })
else:
    checks.append({
        "check": "deriva",
        "resultado": "GO",
        "detalle": f"Deriva estimada {deriva_m:.0f} m",
    })

return {
    "resultado_global": "GO" if go else "NO-GO",
    "hora_evaluacion": datetime.now(timezone.utc).isoformat(),
    "clima": clima,
    "checks": checks,
    "recomendacion": (
        "Proceder con precaucion" if go else "No aplicar en estas condiciones"
    ),
}

def _entry_hash(detalle: dict, prev_hash: str | None) -> str:
    raw = json.dumps(
        {"detalle": detalle, "prev_hash": prev_hash},
        ensure_ascii=False,
        sort_keys=True,
        default=str,
    )
    return hashlib.sha256(raw.encode("utf-8")).hexdigest()

def _build_pdf(path: pathlib.Path, req: PreAplicacionRequest, detalle: dict) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    styles = getSampleStyleSheet()
    doc = SimpleDocTemplate(str(path), pagesize=A4)
    story = [
        Paragraph("Evaluacion pre-aplicacion GO/NO-GO", styles["Title"]),
        Spacer(1, 12),
        Paragraph(f"Producto: {req.producto}", styles["Normal"]),
        Paragraph(f"Cultivo: {req.cultivo or '-'}", styles["Normal"]),
        Paragraph(f"Resultado global: {detalle['resultado_global']}", styles["Heading2"]),
        Paragraph(f"Recomendacion: {detalle['recomendacion']}", styles["Normal"]),
        Spacer(1, 12),
    ]

    rows = [{"Check", "Resultado", "Detalle"}]
    rows.extend(
        [c["check"], c["resultado"], c["detalle"]]
        for c in detalle["checks"]
    )
    table = Table(rows, colWidths=[95, 90, 310])
    table.setStyle(
        TableStyle(
            [
                ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#dcfce7")),
                ("GRID", (0, 0), (-1, -1), 0.35, colors.HexColor("#94a3b8")),
                ("VALIGN", (0, 0), (-1, -1), "TOP"),
                ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
            ]
        )
    )
    story.append(table)
    story.append(Spacer(1, 12))
    story.append(Paragraph("Clima consultado", styles["Heading2"]))
    story.append(Paragraph(json.dumps(detalle["clima"], ensure_ascii=False), styles["Normal"]))
    story.append(Spacer(1, 12))

```

```

story.append(
    Paragraph(
        "Documento generado por AgroAid. No reemplaza criterio profesional, "
        "etiqueta oficial ni normativa local vigente.",
        styles["Italic"],
    )
)
doc.build(story)

def _insert_pre_aplicacion(
    req: PreAplicacionRequest,
    detalle: dict,
    current_user: dict,
    pdf_path: str | None,
) -> dict:
    tenant_id = _tenant_id(current_user)
    user_id = _user_id(current_user)
    prev = db_fetch_one(
        """
        SELECT entry_hash
        FROM pre_aplicaciones
        WHERE tenant_id = %s
        ORDER BY id DESC
        LIMIT 1
        """,
        (tenant_id, ),
    )
    prev_hash = prev["entry_hash"] if prev else None
    entry_hash = _entry_hash(detalle, prev_hash)

    with get_conn() as conn:
        try:
            with conn.cursor(cursor_factory=RealDictCursor) as cur:
                cur.execute(
                    """
                    INSERT INTO pre_aplicaciones
                    (tenant_id, user_id, lat, lon, producto, dosis_l_ha,
                     cultivo, distancia_agua_m, hora_aplicacion, tipo_equipo,
                     resultado_global, detalle, pdf_path, entry_hash, prev_hash)
                    VALUES
                    (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
                     %s, %s, %s, %s, %s)
                    RETURNING *
                    """,
                    (
                        tenant_id,
                        user_id,
                        req.lat,
                        req.lon,
                        req.producto,
                        req.dosis_l_ha,
                        req.cultivo,
                        req.distancia_agua_m,
                        req.hora_aplicacion,
                        req.tipo_equipo,
                        detalle["resultado_global"],
                        json.dumps(detalle, ensure_ascii=False, default=str),
                        pdf_path,
                        entry_hash,
                        prev_hash,
                    ),
                )
            row = dict(cur.fetchone())
            conn.commit()
            return row
        except Exception:
            conn.rollback()

```

```

        raise

@router.post("/pre-aplicacion")
async def crear_pre_aplicacion(
    req: PreAplicacionRequest,
    current_user: dict = Depends(require_any),
):
    detalle = await evaluar_pre_aplicacion(req)
    tenant_slug = _tenant_slug(current_user)

    row = _insert_pre_aplicacion(req, detalle, current_user, None)
    pdf_rel = pathlib.Path("data") / tenant_slug / "prevuelo" / f"pre_aplicacion_{row['id']}.pdf"
    _build_pdf(pdf_rel, req, detalle)

    with get_conn() as conn:
        try:
            with conn.cursor() as cur:
                cur.execute(
                    "UPDATE pre_aplicaciones SET pdf_path = %s WHERE id = %s",
                    (str(pdf_rel), row["id"]),
                )
            conn.commit()
        except Exception:
            conn.rollback()
            raise

    row["pdf_path"] = str(pdf_rel)
    row["pdf_url"] = f"/api/pre-aplicacion/{row['id']}/pdf"
    return {"evaluacion": detalle, "registro": row}

@router.get("/pre-aplicacion/{pre_aplicacion_id}/pdf")
async def descargar_pre_aplicacion_pdf(
    pre_aplicacion_id: int,
    current_user: dict = Depends(require_any),
):
    row = db_fetch_one(
        """
        SELECT pdf_path
        FROM pre_aplicaciones
        WHERE id = %s
        AND tenant_id = %s
        """,
        (pre_aplicacion_id, _tenant_id(current_user)),
    )
    if not row or not row.get("pdf_path"):
        raise HTTPException(status_code=404, detail="PDF no encontrado")

    path = pathlib.Path(row["pdf_path"])
    if not path.exists():
        raise HTTPException(status_code=404, detail="Archivo PDF no encontrado")

    return FileResponse(str(path), media_type="application/pdf", filename=path.name)

```

frontend/widget.js

```
class AgroAidWidget extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: "open" });
  }

  connectedCallback() {
    const apiKey = this.getAttribute("api-key") || "";
    const baseUrl =
      this.getAttribute("base-url") || window.location.origin;

    this.shadowRoot.innerHTML = `
<style>
:host {
  display: block;
  max-width: 420px;
  font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: #0f172a;
}
.widget {
  border: 1px solid #bbf7d0;
  border-radius: 8px;
  background: #f8fafc;
  padding: 14px;
  box-shadow: 0 12px 30px rgba(15, 23, 42, 0.08);
}
.title {
  margin: 0 0 8px;
  font-size: 16px;
  font-weight: 800;
  color: #14532d;
}
textarea {
  width: 100%;
  min-height: 96px;
  resize: vertical;
  box-sizing: border-box;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  padding: 9px;
  font: inherit;
}
button {
  margin-top: 10px;
  width: 100%;
  border: 0;
  border-radius: 6px;
  background: #16a34a;
  color: white;
  padding: 10px 12px;
  font-weight: 800;
  cursor: pointer;
}
button:disabled {
  cursor: wait;
  opacity: 0.7;
}
.resultado {
  margin-top: 10px;
  padding: 10px;
  border-radius: 6px;
  background: white;
  border: 1px solid #e2e8f0;
  line-height: 1.4;
  min-height: 24px;

```

```

    }
</style>
<div class="widget">
  <p class="title">AgroAid Risk Score</p>
  <textarea id="q" placeholder="Consulta sobre seguridad agricola"></textarea>
  <button id="btn">Evaluar riesgo</button>
  <div id="resultado" class="resultado"></div>
</div>
`;

const btn = this.shadowRoot.getElementById("btn");
const qInput = this.shadowRoot.getElementById("q");
const resultado = this.shadowRoot.getElementById("resultado");

btn.addEventListener("click", async () => {
  const q = qInput.value.trim();
  if (!q) return;

  btn.disabled = true;
  resultado.textContent = "Evaluando...";

  try {
    const res = await fetch(
      `${baseUrl}/api/risk-score?q=${encodeURIComponent(q)}`,
      { headers: { "X-API-Key": apiKey } }
    );

    if (!res.ok) {
      throw new Error(await res.text());
    }

    const data = await res.json();
    resultado.innerHTML = `
      <strong>${data.nivel_riesgo} || "SIN NIVEL"</strong><br>
      ${data.justificacion} || "Sin justificacion disponible"
    `;
  } catch (error) {
    resultado.textContent = "No se pudo evaluar la consulta.";
    console.error(error);
  } finally {
    btn.disabled = false;
  }
});
}

customElements.define("agroaid-widget", AgroAidWidget);

```

frontend/widget-demo.html

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Demo Widget AgroAid</title>
  <style>
    body {
      margin: 0;
      min-height: 100vh;
      display: grid;
      place-items: center;
      background: #eef2f7;
      font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
    }
    main {
      width: min(460px, calc(100vw - 32px));
    }
  </style>
</head>
<body>
  <main>
    <script src="/widget.js"></script>
    <agroaid-widget
      api-key="ags_su_api_key_aqui"
      base-url="">
    </agroaid-widget>
  </main>
</body>
</html>
```

core/pdf_receta.py

```
from __future__ import annotations

import pathlib
import re

from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle

from core.database import db_fetch_one


def _safe_slug(value: str) -> str:
    value = (value or "default").strip().lower()
    return re.sub(r"^[^a-z0-9_-]+", "-", value).strip("-") or "default"


def _text(value) -> str:
    return "" if value is None else str(value)


def crear_pdf_receta(receta: dict, tenant_slug: str = "default") -> str:
    safe_slug = _safe_slug(tenant_slug)
    numero = receta["numero_receta"]
    out_dir = pathlib.Path("data") / safe_slug / "recetas"
    out_dir.mkdir(parents=True, exist_ok=True)
    out_path = out_dir / f"{numero}.pdf"

    branding = db_fetch_one(
        """
        SELECT app_name, footer_text, primary_color, accent_color, logo_url
        FROM tenant_branding
        WHERE tenant_id = %s
        """
    ) or {}

    app_name = branding.get("app_name") or "AgroAid"
    footer = (
        "Documento generado por AgroAid - "
        "NO reemplaza firma profesional habilitada"
    )
    primary = branding.get("primary_color") or "#16a34a"

    styles = getSampleStyleSheet()
    doc = SimpleDocTemplate(
        str(out_path),
        pagesize=A4,
        rightMargin=42,
        leftMargin=42,
        topMargin=42,
        bottomMargin=42,
    )

    story = [
        Paragraph(app_name, styles["Title"]),
        Paragraph("Receta agronomica electronica", styles["Heading1"]),
        Spacer(1, 10),
        Paragraph(f"Numero: <b>{numero}</b>", styles["Normal"]),
        Paragraph(f"Estado: <b>{receta.get('estado', 'borrador')}</b>", styles["Normal"]),
        Paragraph(f"Fecha de emision: {_text(receta.get('created_at'))}", styles["Normal"]),
        Spacer(1, 12),
    ]
    rows = [
```

```

["Campo", "Valor"],
["Producto", _text(receta.get("producto"))],
["Principio activo", _text(receta.get("principio_activo"))],
["Cultivo", _text(receta.get("cultivo"))],
["Lote", _text(receta.get("lote"))],
["Superficie", f"{{_text(receta.get('superficie_ha'))}} ha"],
["Dosis", _text(receta.get("dosis"))],
["Volumen de agua", _text(receta.get("volumen_agua"))],
["Fecha de aplicacion", _text(receta.get("fecha_aplicacion"))],
["Nivel de riesgo", _text(receta.get("nivel_riesgo"))],
["Observaciones", _text(receta.get("observaciones"))],
]

table = Table(rows, colWidths=[150, 340])
table.setStyle(
    TableStyle(
        [
            ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor(primary)),
            ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
            ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
            ("GRID", (0, 0), (-1, -1), 0.35, colors.HexColor("#cbd5e1")),
            ("BACKGROUND", (0, 1), (0, -1), colors.HexColor("#f1f5f9")),
            ("VALIGN", (0, 0), (-1, -1), "TOP"),
            ("LEFTPADDING", (0, 0), (-1, -1), 7),
            ("RIGHTPADDING", (0, 0), (-1, -1), 7),
            ("TOPPADDING", (0, 0), (-1, -1), 6),
            ("BOTTOMPADDING", (0, 0), (-1, -1), 6),
        ]
    )
)
story.append(table)
story.append(Spacer(1, 18))
story.append(Paragraph(footer, styles["Italic"]))
story.append(Paragraph(_text(branding.get("footer_text") or ""), styles["Normal"]))

doc.build(story)
return str(out_path)

```

routers/recetas.py

```
from __future__ import annotations

import pathlib
from datetime import date

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import FileResponse
from psycpg2.extras import RealDictCursor
from pydantic import BaseModel, Field

from core.database import db_fetch_all, db_fetch_one, get_conn
from core.deps import require_admin, require_any
from core.pdf_receta import crear_pdf_receta

router = APIRouter(prefix="/api/recetas", tags=["Recetas agronomicas"])

class RecetaCreate(BaseModel):
    consulta_id: int | None = None
    producto: str
    principio_activo: str | None = None
    cultivo: str
    lote: str | None = None
    superficie_ha: float | None = Field(default=None, ge=0)
    dosis: str
    volumen_agua: str | None = None
    fecha_aplicacion: date | None = None
    observaciones: str | None = None
    nivel_riesgo: str | None = None

def _tenant_id(current_user: dict) -> int:
    return int(current_user.get("tenant_id") or 1)

def _user_id(current_user: dict) -> int | None:
    raw = current_user.get("id")
    return int(raw) if raw is not None else None

def _tenant_slug(current_user: dict) -> str:
    return (current_user.get("tenant_slug") or "default").strip() or "default"

def _next_numero_receta(cur) -> tuple[int, str]:
    cur.execute("SELECT nextval('recetas_agronicas_id_seq')")
    receta_id = int(cur.fetchone()[0])
    numero = f"RA-{date.today().year}-{receta_id:06d}"
    return receta_id, numero

def _insert_receta(
    req: RecetaCreate,
    current_user: dict,
    estado: str = "borrador",
) -> dict:
    tenant_id = _tenant_id(current_user)
    user_id = _user_id(current_user)

    with get_conn() as conn:
        try:
            with conn.cursor(cursor_factory=RealDictCursor) as cur:
                receta_id, numero = _next_numero_receta(cur)
                cur.execute(
```

```

        """
        INSERT INTO recetas_agronomicas
            (id, tenant_id, user_id, consulta_id, numero_receta,
             producto, principio_activo, cultivo, lote, superficie_ha,
             dosis, volumen_agua, fecha_aplicacion, observaciones,
             nivel_riesgo, estado)
        VALUES
            (%s, %s, %s, %s, %s,
             %s, %s, %s, %s, %s,
             %s, %s, %s, %s,
             %s, %s)
        RETURNING *
        """
        (
            receta_id,
            tenant_id,
            user_id,
            req.consulta_id,
            numero,
            req.producto,
            req.principio_activo,
            req.cultivo,
            req.lote,
            req.superficie_ha,
            req.dosis,
            req.volumen_agua,
            req.fecha_aplicacion,
            req.observaciones,
            req.nivel_riesgo,
            estado,
        ),
    )
    receta = dict(cur.fetchone())
    conn.commit()
    return receta
except Exception:
    conn.rollback()
    raise


def _update_pdf_path(receta_id: int, tenant_id: int, pdf_path: str) -> None:
    with get_conn() as conn:
        try:
            with conn.cursor() as cur:
                cur.execute(
                    """
                    UPDATE recetas_agronomicas
                    SET pdf_path = %s
                    WHERE id = %s
                    AND tenant_id = %s
                    """
                    ,
                    (pdf_path, receta_id, tenant_id),
                )
            conn.commit()
        except Exception:
            conn.rollback()
            raise


def create_receta_borrador(
    current_user: dict,
    producto: str,
    cultivo: str,
    dosis: str,
    consulta_id: int | None = None,
    principio_activo: str | None = None,
    nivel_riesgo: str | None = None,
    observaciones: str | None = None,

```

```

) -> dict:
    req = RecetaCreate(
        consulta_id=consulta_id,
        producto=producto,
        principio_activo=principio_activo,
        cultivo=cultivo,
        dosis=dosis,
        nivel_riesgo=nivel_riesgo,
        observaciones=observaciones,
    )
    receta = _insert_receta(req, current_user, estado="borrador")
    pdf_path = crear_pdf_receta(receta, _tenant_slug(current_user))
    _update_pdf_path(receta["id"], receta["tenant_id"], pdf_path)
    receta["pdf_path"] = pdf_path
    receta["pdf_url"] = f"/api/recetas/{receta['id']}/pdf"
    return receta


@router.post("")
async def crear_receta(
    req: RecetaCreate,
    current_user: dict = Depends(require_any),
):
    receta = _insert_receta(req, current_user, estado="borrador")
    pdf_path = crear_pdf_receta(receta, _tenant_slug(current_user))
    _update_pdf_path(receta["id"], receta["tenant_id"], pdf_path)

    receta["pdf_path"] = pdf_path
    receta["pdf_url"] = f"/api/recetas/{receta['id']}/pdf"
    return receta


@router.get("")
async def listar_recetas(
    current_user: dict = Depends(require_any),
):
    return db_fetch_all(
        """
        SELECT id, numero_receta, producto, principio_activo, cultivo, lote,
            superficie_ha, dosis, fecha_aplicacion, nivel_riesgo, estado,
            pdf_path, created_at
        FROM recetas_agronomicas
        WHERE tenant_id = %s
        ORDER BY created_at DESC
        LIMIT 200
        """,
        (_tenant_id(current_user),),
    )


@router.get("/{receta_id}/pdf")
async def descargar_pdf_receta(
    receta_id: int,
    current_user: dict = Depends(require_any),
):
    row = db_fetch_one(
        """
        SELECT pdf_path, numero_receta
        FROM recetas_agronomicas
        WHERE id = %s
            AND tenant_id = %s
        """,
        (receta_id, _tenant_id(current_user)),
    )

    if not row or not row.get("pdf_path"):
        raise HTTPException(status_code=404, detail="PDF no encontrado")
    path = pathlib.Path(row["pdf_path"])

```

```

    if not path.exists():
        raise HTTPException(status_code=404, detail="Archivo PDF no encontrado")

    return FileResponse(
        str(path),
        media_type="application/pdf",
        filename=f"{row['numero_receta']}.pdf",
    )

def _cambiar_estado(receta_id: int, current_user: dict, estado: str) -> dict:
    row = db_fetch_one(
        """
        UPDATE recetas_agronomicas
        SET estado = %s
        WHERE id = %s
        AND tenant_id = %s
        RETURNING *
        """
        ,
        (estado, receta_id, _tenant_id(current_user)),
    )
    if not row:
        raise HTTPException(status_code=404, detail="Receta no encontrada")
    return row

@router.put("/{receta_id}/emitir")
async def emitir_receta(
    receta_id: int,
    current_user: dict = Depends(require_admin),
):
    return _cambiar_estado(receta_id, current_user, "emitida")

@router.put("/{receta_id}/anular")
async def anular_receta(
    receta_id: int,
    current_user: dict = Depends(require_admin),
):
    return _cambiar_estado(receta_id, current_user, "anulada")

```

routers/imagen.py - integracion opcional con recetas

```
# Dentro de consulta_imagen(), despues de:
#
#     evaluacion = await iniciar_consulta(req, current_user)
#
# agregar:

receta = None
try:
    producto_receta = label_data.get("producto")
    dosis_receta = label_data.get("dosis_recomendada")
    cultivos = label_data.get("cultivos_o_especies_permitidos") or []
    cultivo_receta = cultivos[0] if cultivos else "cultivo no especificado"

    if producto_receta and dosis_receta:
        from routers.recetas import create_receta_borrador

        receta = create_receta_borrador(
            current_user=current_user,
            consulta_id=evaluacion.get("consulta_id"),
            producto=producto_receta,
            principio_activo=label_data.get("principio_activo"),
            cultivo=cultivo_receta,
            dosis=dosis_receta,
            nivel_riesgo=evaluacion.get("nivel_riesgo"),
            observaciones="Borrador generado desde analisis de etiqueta.",
        )
except Exception:
    receta = None

# Y en el return agregar:
#
#     "receta_id": receta["id"] if receta else None,
#     "receta": receta,
```

app.py - imports, routers y archivos frontend

1) Cambiar el import de responses para incluir Response:

```
from fastapi.responses import FileResponse, Response, StreamingResponse
```

2) Agregar estos imports junto a los otros routers:

```
from routers.mapa import router as mapa_router
from routers.prevuelo import router as prevuelo_router
from routers.recetas import router as recetas_router
```

3) Registrar los routers junto a los include_router existentes:

```
app.include_router(mapa_router)
app.include_router(prevuelo_router)
app.include_router(recetas_router)
```

4) Agregar estos endpoints despues de FRONTEND_DIR:

```
@app.get("/mapa")
async def mapa_page():
    page = FRONTEND_DIR / "mapa.html"
    if page.exists():
        return FileResponse(str(page))
    return FileResponse(str(FRONTEND_DIR / "index.html"))
```

```
@app.get("/widget-demo")
async def widget_demo_page():
    page = FRONTEND_DIR / "widget-demo.html"
    if page.exists():
        return FileResponse(str(page))
    return FileResponse(str(FRONTEND_DIR / "index.html"))
```

```
@app.get("/widget.js", response_class=Response)
async def widget_js():
    path = FRONTEND_DIR / "widget.js"
    if not path.exists():
        return Response(
            content="console.error('frontend/widget.js no encontrado');",
            media_type="application/javascript",
            status_code=404,
        )

    return Response(
        content=path.read_text(encoding="utf-8"),
        media_type="application/javascript",
    )
```

Pruebas manuales rapidas

```
# Mapa: crear evento
curl -X POST http://localhost:8000/api/mapa/eventos \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"lat":-34.6,"lon":-58.4,"tipo":"plaga","descripcion":"manchas en hojas"}'

# Mapa: listar eventos cercanos
curl "http://localhost:8000/api/mapa/eventos?lat=-34.6&lon=-58.4&radio_km=100" \
-H "Authorization: Bearer <token>"

# Pre-vuelo
curl -X POST http://localhost:8000/api/pre-aplicacion \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{
  "lat": -34.6,
  "lon": -58.4,
  "producto": "Glifosato 48%",
  "dosis_l_ha": 3.0,
  "cultivo": "soja",
  "distancia_agua_m": 150,
  "hora_aplicacion": "2026-09-15T08:00:00",
  "tipo_equipo": "drone",
  "epp_disponible": ["guantes", "mascara"]
}'

# Recetas
curl -X POST http://localhost:8000/api/recetas \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{
  "producto": "Glifosato 48%",
  "principio_activo": "Glifosato",
  "cultivo": "soja",
  "lote": "Lote Norte 3",
  "superficie_ha": 50.5,
  "dosis": "2.5 L/ha",
  "fecha_aplicacion": "2026-09-20"
}'
```